



CIN0136

Desenvolvimento de Software

Desenvolvimento de Software

Prof. Vinicius Cardoso Garcia

vcg@cin.ufpe.br :: @vinicius3w :: assertlab.com

<http://viniciusgarcia.me>



**Centro de
Informática**
UFPE



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO

Licença do material

Este Trabalho foi licenciado com uma Licença



**Atribuição-NãoComercial-
Compartilha Igual 4.0 Internacional
(CC BY-NC-SA 4.0)**

Mais informações visite

<https://creativecommons.org/licenses/by-nc-sa/4.0/deed.pt> BR



Engenharia de Software em Dimensões

É um material abrangente, didático e atualizado que apresenta a Engenharia de Software como uma disciplina multidimensional. Cada capítulo foca em uma dimensão prática, conectando teoria e realidade profissional com clareza, profundidade e propósito.

Prof. Vinicius Cardoso Garcia

<https://bit.ly/esdbook>

Engenharia de Software em Dimensões

Da base conceitual à prática profissional



Ponto de Commit

O podcast figital da disciplina (IF977) Engenharia de Software do curso de bacharelado em Sistemas de Informação do Centro de Informática (CIn) da Universidade Federal de Pernambuco (UFPE).

<https://go.uaite.xyz/ponto-de-commit>

Ponto de Commit



O podcast figital da disciplina de Engenharia de Software



Engenharia de Software Moderna

Engenharia de Software Moderna é um livro destinado a alunos de cursos de graduação em Computação. Ele pode ser lido também por profissionais que buscam um conhecimento horizontal da área.

Prof. Marco Tulio Valente

<https://engsoftmoderna.info>

@engsoftmoderna

ENGENHARIA — DE — SOFTWARE MODERNA

Princípios e práticas para
desenvolvimento de software
com produtividade



MARCO TULIO VALENTE



Do caos ao processo: modelos de desenvolvimento de software

Modelos de desenvolvimento de software,
e por que eles existem



O que você vai aprender hoje

Esta aula não é sobre memorizar modelos — é sobre **criar a necessidade deles**. Você já viveu algo parecido com um processo na semana passada. Hoje vamos dar nome a isso e expandir o horizonte histórico.

01

Processo como resposta humana

Entender que processo resolve um problema de coordenação, não um problema técnico

03

Incremental vs. Iterativo

Distinguir os dois modelos e identificar as vantagens de cada um

02

Cascata: reabilitação crítica

Compreender por que o cascata surgiu como solução racional — antes de criticá-lo

04

Seu projeto no espelho

Reconhecer no projeto em andamento elementos de um processo estruturado



Começa com a semana passada

O que aconteceu na quinta? Alguém pode me descrever o que aconteceu, em ordem, desde o momento em que a aula começou?

Vocês seguiram um processo, mesmo que ninguém tenha chamado assim. Tinham uma ordem implícita. O que teria acontecido se cada um tivesse feito de um jeito completamente diferente, sem nenhuma estrutura?



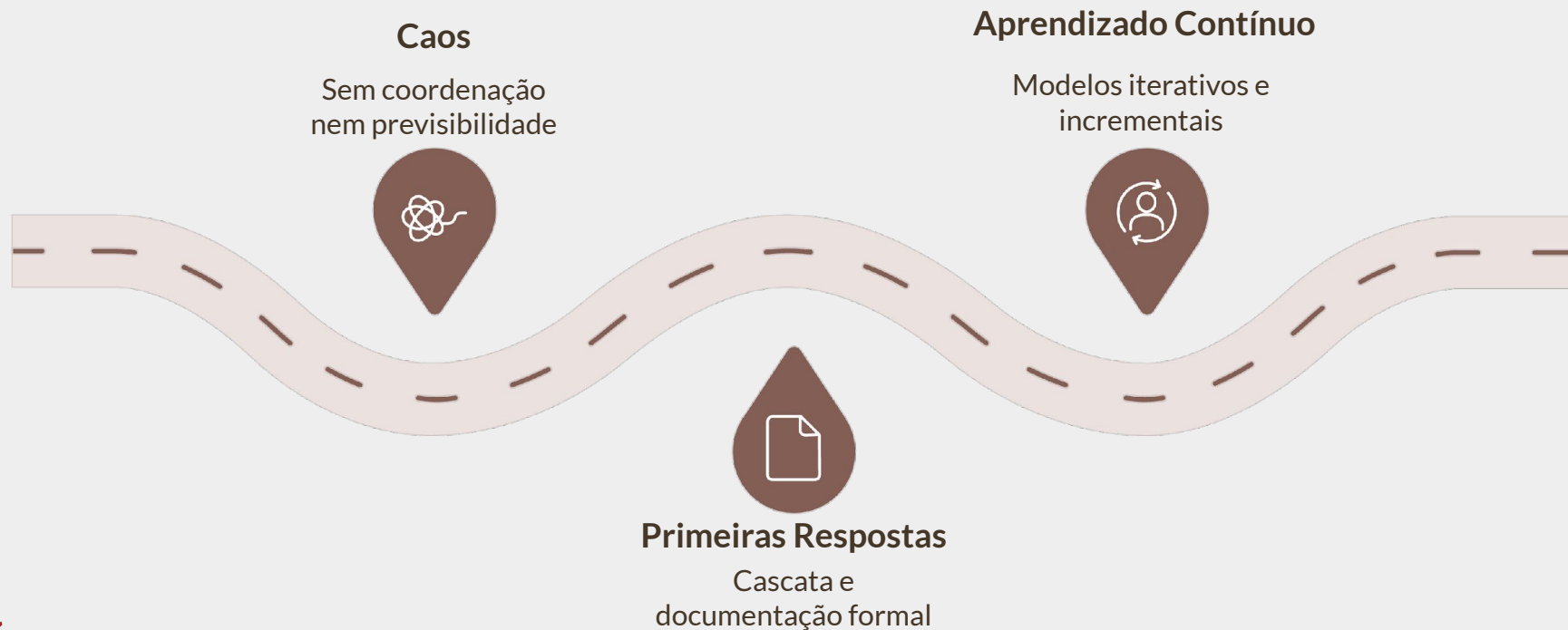


Processo é a resposta que a engenharia de software encontrou para um problema que é humano, não técnico: como coordenar várias pessoas trabalhando num produto complexo ao longo do tempo.



Por que processo existe?

E essa resposta evoluiu. Desde os anos 1970 até hoje, cada novo modelo surgiu para resolver limitações do anterior. Vamos percorrer essa história — não como curiosidade histórica, mas como mapa para entender onde seu projeto se encaixa.



Poor President Obama...

HealthCare.gov Learn Get Insurance Log in Español

Individuals & Families Small Businesses All Topics ▾ Search SEARCH

The System is down at the moment.

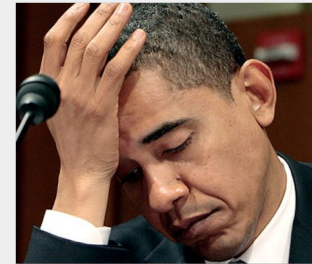
We're working to resolve the issue as soon as possible. Please try again later.

Please include the reference ID below if you wish to contact us at 1-800-318-2596 for support.

Error from: https%3A//www.healthcare.gov/marketplace/global/en_US/registration%23signUpStepOne
Reference ID: 0.cdd73b17.1380636213.edae7e9

Health Insurance Marketplace **181** DAYS LEFT TO ENROLL

OCT	Open Enrollment Began	JAN	Coverage Can Begin	MAR	Open Enrollment Closes
1		1		31	



lições do healthcare.gov

1. sprints + iterations $\neq$ agile
healthcare.gov nunca foi testado de fato
2. um 'sistema' RESULTA de um 'SISTEMA'
{não havia como 'parar' healthcare.gov...}
3. TESTE tem que ser parte da ENTREGA
e não apagar incêndios depois da 'entrega'



Como lidar com mudanças?

“Planeje jogar [implementação] fora; porque você irá, de qualquer maneira.”

Fred Brooks, Jr. (1999 Turing Award winner)

Muitas vezes, após a construção da primeira versão, os desenvolvedores aprendem a maneira correta que deveriam ter construído.



(Photo by Carola Lauber of SD&M www.sdm.de. Used by permission under CC-BY-SA-3.0.)





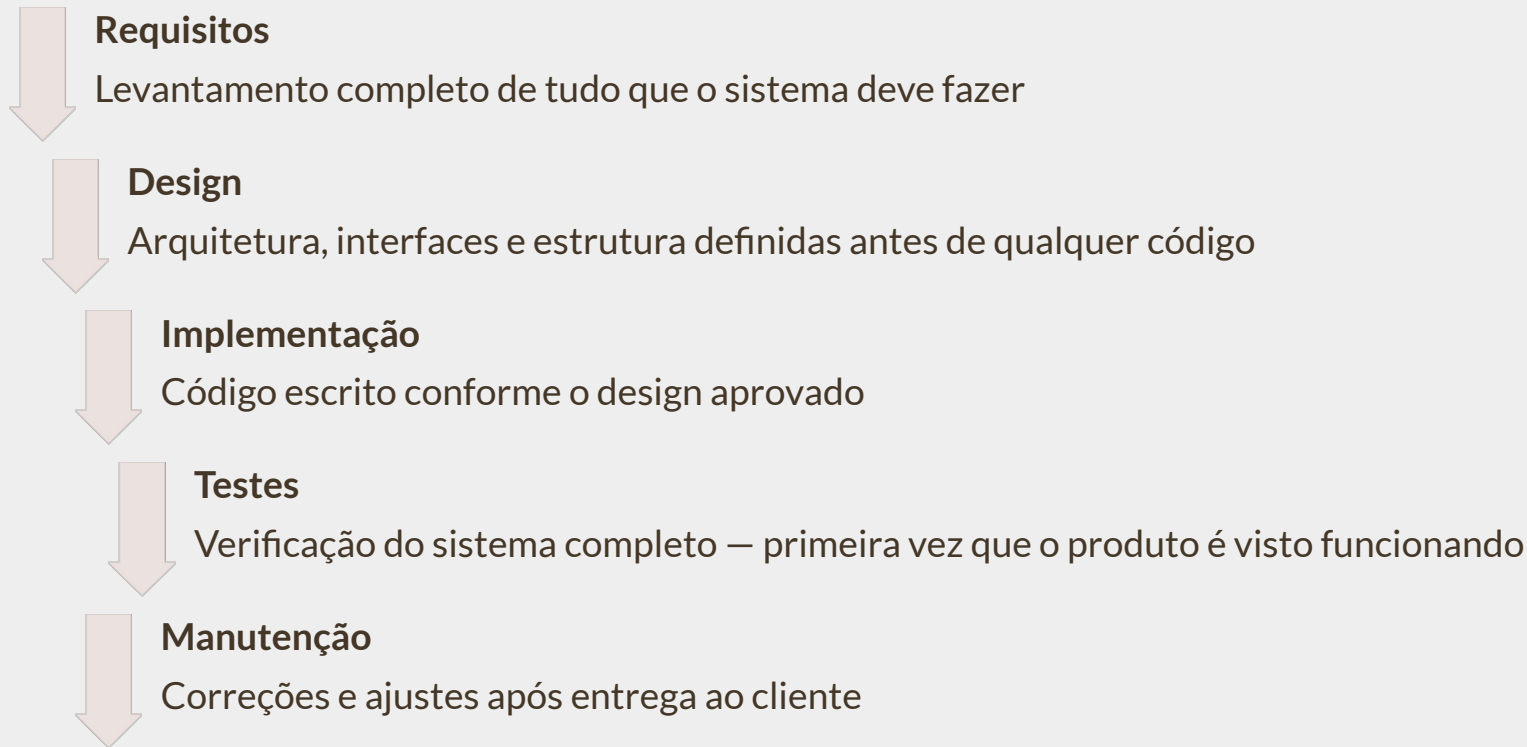
1970: o nascimento do cascata

Winston Royce formalizou o modelo cascata num contexto muito específico: software era desenvolvido como hardware. Você planejava tudo antes de construir, porque mudar era caro ou impossível.



A lógica do cascata

Cada fase só começa quando a anterior está completamente encerrada.



O cascata é realmente ruim, ou apenas mal compreendido?

O cascata foi formalizado por Winston Royce em 1970. Na época, software era desenvolvido como hardware, você planejava tudo antes de construir qualquer coisa, porque mudar era caro ou impossível. Imaginem construir um foguete: você não vai na metade e decide mudar o motor. O cascata fazia sentido para aquele contexto.



O cascata é realmente ruim, ou apenas mal compreendido?

O argumento do foguete

Imaginem construir um foguete: você não vai na metade e decide mudar o motor. Em contextos onde mudança tem custo proibitivo ou risco de vida, planejar tudo antes faz sentido absoluto.

O cascata foi racional para o seu tempo e contexto, não foi ingenuidade.

O problema real

A crítica ao cascata não é que ele seja burro — é que foi **aplicado em contextos para os quais não foi projetado**.

Quando o contexto muda (usuários que mudam de ideia, requisitos incertos, times pequenos), o processo precisa mudar junto.



As perguntas que revelam as limitações

Três perguntas concretas que desarmam qualquer romantismo excessivo com o cascata:

? O cliente muda de ideia

O que acontece quando o cliente altera os requisitos *depois* que a fase de requisitos foi formalmente encerrada e assinada?

? Bug descoberto nos testes

O que acontece quando um erro descoberto na fase de testes exige mudar uma decisão tomada ainda na fase de design?

? Quando o usuário vê o produto?

Em um projeto cascata, quando os usuários finais veem e usam o produto pela primeira vez — e o que acontece se não gostarem?

Essas perguntas conduzem à mesma conclusão: **descoberta tardia de problemas, custo altíssimo de mudança, entrega de software que não serve ao usuário real.**





Em que contextos o cascata ainda faz sentido?

Projetos com Requisitos Estáveis

Ideal para projetos onde os requisitos são bem compreendidos desde o início e têm baixa probabilidade de mudança durante o desenvolvimento.

Sistemas Críticos

Adequado para sistemas onde a segurança é crítica, como software médico ou aeroespacial, devido à sua ênfase em documentação detalhada e revisões rigorosas.

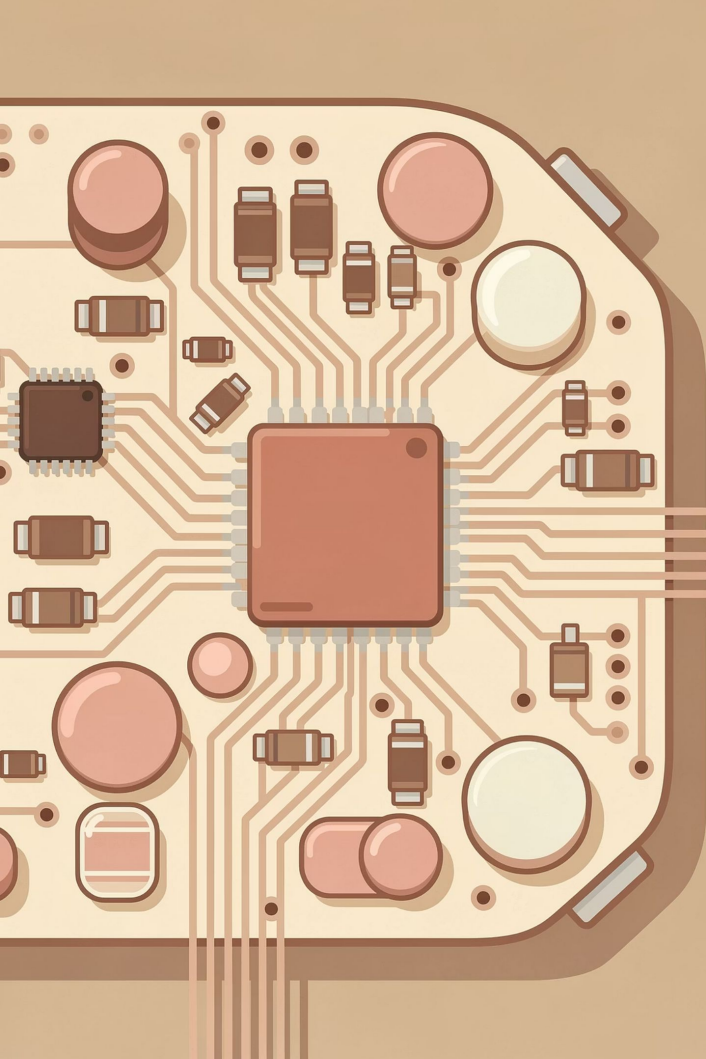
Projetos de Curto Prazo

Eficaz para projetos pequenos e de curta duração, onde o escopo é bem definido e o tempo de desenvolvimento é limitado.

Migração de Sistemas Legados

Útil em projetos de migração de sistemas antigos, onde os requisitos são claros e baseados no sistema existente.





Em que contextos o cascata ainda faz sentido?

"Mas o cascata ainda existe em produção — e com razão." Peça exemplos à turma antes de revelar os contextos.



Aviação e Defesa

Sistemas embarcados críticos onde uma falha é inaceitável e a especificação não pode mudar após a certificação



Dispositivos Médicos

Marca-passos, sistemas de controle cirúrgico, qualquer software onde mudança durante o projeto pode custar vidas



Contratos Governamentais

Projetos com especificação totalmente fechada em contrato, onde o escopo não pode ser alterado sem aprovação formal

- ☐ Quando o problema muda, o processo precisa mudar junto. O cascata não foi abolido, foi recontextualizado.

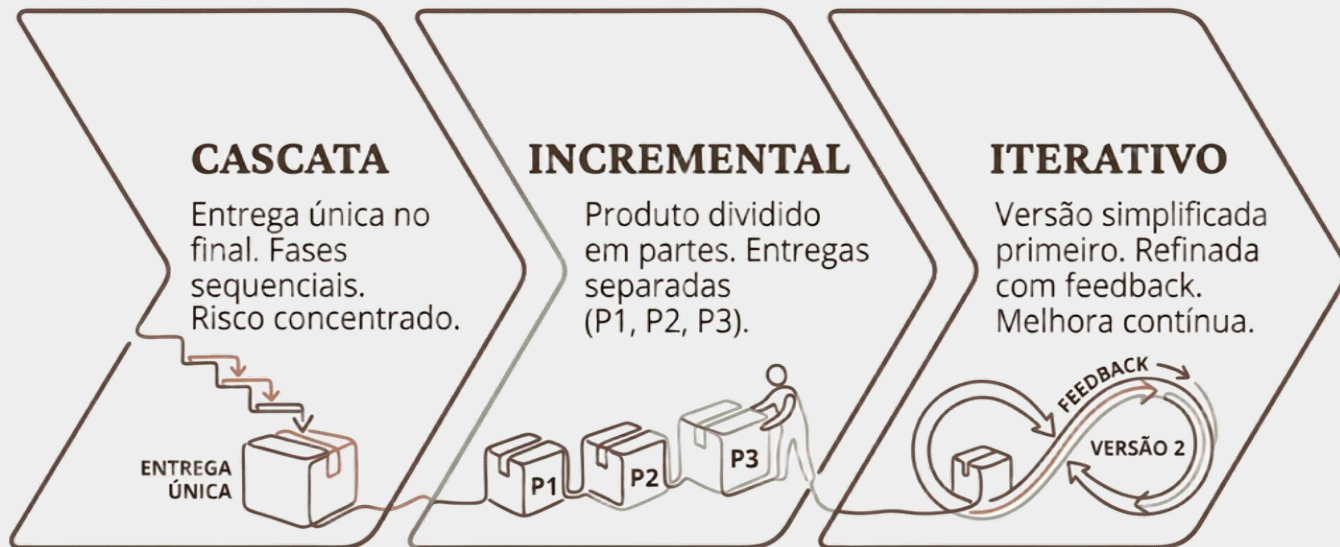


1. sprints + iterations ≠ agile
healthcare.gov nunca foi testado de fato
2. um 'sistema' RESULTA de um 'SISTEMA'
{não havia como 'parar' healthcare.gov...}
3. TESTE tem que ser parte da ENTREGA
e não apagar incêndios depois da 'entrega'
4. de nada adianta INFORMATIZAR O CAOS
se os processos não estão bem definidos...
5. não há sistema PERFEITO; nem POR LEI...
governos tendem a insistir nisso; **#FAIL!**...
6. sistemas -online, high profile- são ALVOS
healthcare.gov nunca foi olhado assim



Três modelos, três filosofias

A diferença não é de nível de modernidade, é de filosofia de entrega e aprendizado.



Modelo Incremental: entrega antes do fim

Como funciona

O produto é dividido em partes funcionais que são entregues uma a uma. O cliente vê resultados reais antes de o projeto terminar. O risco se distribui ao longo do tempo.

O que resolve do cascata

- Feedback mais cedo — partes do sistema já em uso
- Risco distribuído, não concentrado no final
- Valor entregue progressivamente ao stakeholder

O que o incremental não resolve

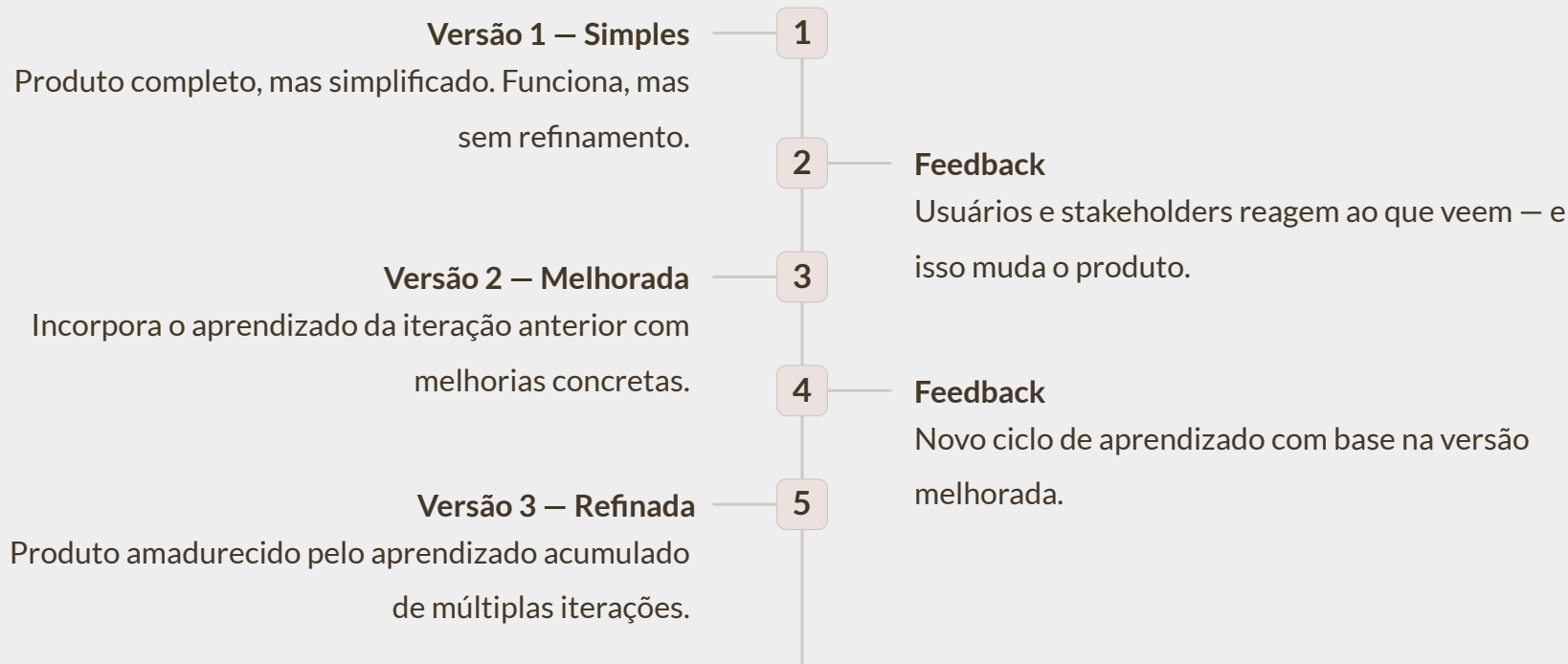
Dentro de cada incremento, o processo pode ainda ser rígido como o cascata.

Se a Parte 1 foi mal especificada, corrigir pode forçar a reconstrução das partes seguintes — o problema se desloca, mas não desaparece.



Modelo Iterativo: aprender é o plano

A diferença fundamental: o iterativo **assume explicitamente que você vai aprender** e que o produto vai mudar. Cada iteração é uma oportunidade planejada de incorporar o que foi aprendido.



A distinção que importa: um exemplo concreto

App de cadastro com três campos: nome, endereço e histórico. Os dois modelos produzem o mesmo produto final, mas de formas completamente diferentes.

Modo Incremental

Semana 1: entrego só o formulário de nome — funcional e completo

Semana 2: entrego o campo de endereço como novo incremento

Semana 3: entrego o histórico — produto completo

Cada parte é autônoma. O produto cresce por **adição**.

Modo Iterativo

Semana 1: entrego versão com os três campos, feia, sem validação, mas funcional

Semana 2: adiciono validação e melhoro a interface com base no feedback

Semana 3: refino usabilidade com base no que os usuários reportaram

O produto inteiro existe desde o início e cresce por **refinamento**.

- ❑ Incremental = produto cresce por adição de partes. Iterativo = produto amadurece por refinamento de versões. São abordagens diferentes para problemas diferentes.



Seu projeto no espelho dos modelos



Olhando para o que vocês já fizeram, o Workshop Ágil, o Kanban, a construção do backlog com o lego4scrum, o que já estamos fazendo que se parece com um processo estruturado?



Reconhecendo o processo que já existe



O Backlog

Planejamento estruturado do que precisa ser feito. Não é uma lista aleatória — é prioridade com intenção. Isso é processo.



O Kanban

Visualização do fluxo de trabalho. Torna visível o que está parado, o que está em progresso e o que foi concluído. Isso é processo.



A Reunião com o Stakeholder

Elicitação de requisitos estruturada. Havia uma ordem, havia um produto (o backlog), havia critérios implícitos. Isso é processo.



A Definition of Done

Critério explícito de conclusão. Transforma "acho que está pronto" em "está pronto quando atende a esses critérios". Isso é processo.





**E o que estamos fazendo
ainda de forma
improvisada, que um
processo mais definido
nos ajudaria a melhorar?**





O que vimos hoje e o que vem a seguir

Processos não são invenções acadêmicas, são respostas a problemas reais de coordenação que toda equipe enfrenta. O cascata foi a primeira resposta séria. O incremental e o iterativo corrigiram suas limitações.



Leitura para terça

Cap. 4, seções 4.3 a 4.5 — Scrum, Kanban, Lean e os critérios para escolher entre eles



Venha preparado para defender

Se vocês fossem escolher um processo para o projeto *hoje*, qual seria e por quê? Pensem na resposta enquanto leem.



contacts

vcg@cin.ufpe.br
[@vinicius3w](https://twitter.com/vinicius3w)
viniciusgarcia.me

academy

go.uaite.xyz/vcg-es
go.uaite.xyz/vcg-microservices
go.uaite.xyz/vcg-devops
go.uaite.xyz/vcg-esaia
go.uaite.xyz/vcg-deso

Desenvolvimento de Software (CIN0136)

Do caos ao processo: modelos de desenvolvimento de software